Tenant-Aware Custom Attributes

Typed EAV with a domain / subdomain catalog and a polymorphic entity link

Overview

This design evolves the typed EAV product-attribute model into a shared attribute service that any table in the application can use, for any number of tenants. Three changes drive it:

- **Attributes are tenant-specific.** Every catalog and value table carries `tenant_id`, and composite foreign keys that lead with it make cross-tenant references impossible. Two tenants can each define their own *color* with different options.
- **Attributes are classified by domain and subdomain.** `attribute_definition` references an `attribute_domain` (SALES, CRM) and optionally an `attribute_subdomain` (PRODUCT_SPECS, LOYALTY), which must belong to that domain.
- **Values attach to any record.** `attribute_value` replaces `product_id` with `entity_type` (which table the record lives in) and `entity_id` (that record's primary key). Products, customers, suppliers or any future table can carry custom attributes without new tables.

Everything that made the original design strong is kept: typed value columns, the one-value CHECK constraint, option lists, multi-valued attributes through `value_seq`, per-type required flags, and the audit history.

Tables at a glance

Table	Scope	Purpose
`tenant`	System	Each business or organization that owns its own attribute catalog.
`entity_type`	System	Kinds of records attributes can attach to (PRODUCT, CUSTOMER, SUPPLIER), with the source table and PK column name.
`attribute_domain`	Tenant	Top-level business area that groups attributes (SALES, CRM).
`attribute_subdomain`	Tenant	Finer grouping inside one domain (SALES > PRODUCT_SPECS).
`attribute_definition`	Tenant	The catalog: code, data type, unit, min/max, max length, regex, multi-valued and filterable flags, domain and subdomain.
`attribute_option`	Tenant	Allowed values for OPTION (dropdown) attributes.
`attribute_entity_type`	Tenant	Which attributes apply to which entity type, with required flag, default, UI group and sort order.
`attribute_value`	Tenant	The values: one typed column per data type, keyed by tenant, entity type, entity ID, attribute and sequence.
`attribute_value_history`	Tenant	Audit trail of every insert, update and delete, written by a trigger.

Entity relationship diagram

Grey tables are system-level, blue tables are the tenant's metadata catalog, and green tables hold the data. The dashed link to the source tables is logical: `entity_id` points at a different table for each `entity_type`, so it cannot be a real foreign key. Triggers and a data-quality query cover that gap.

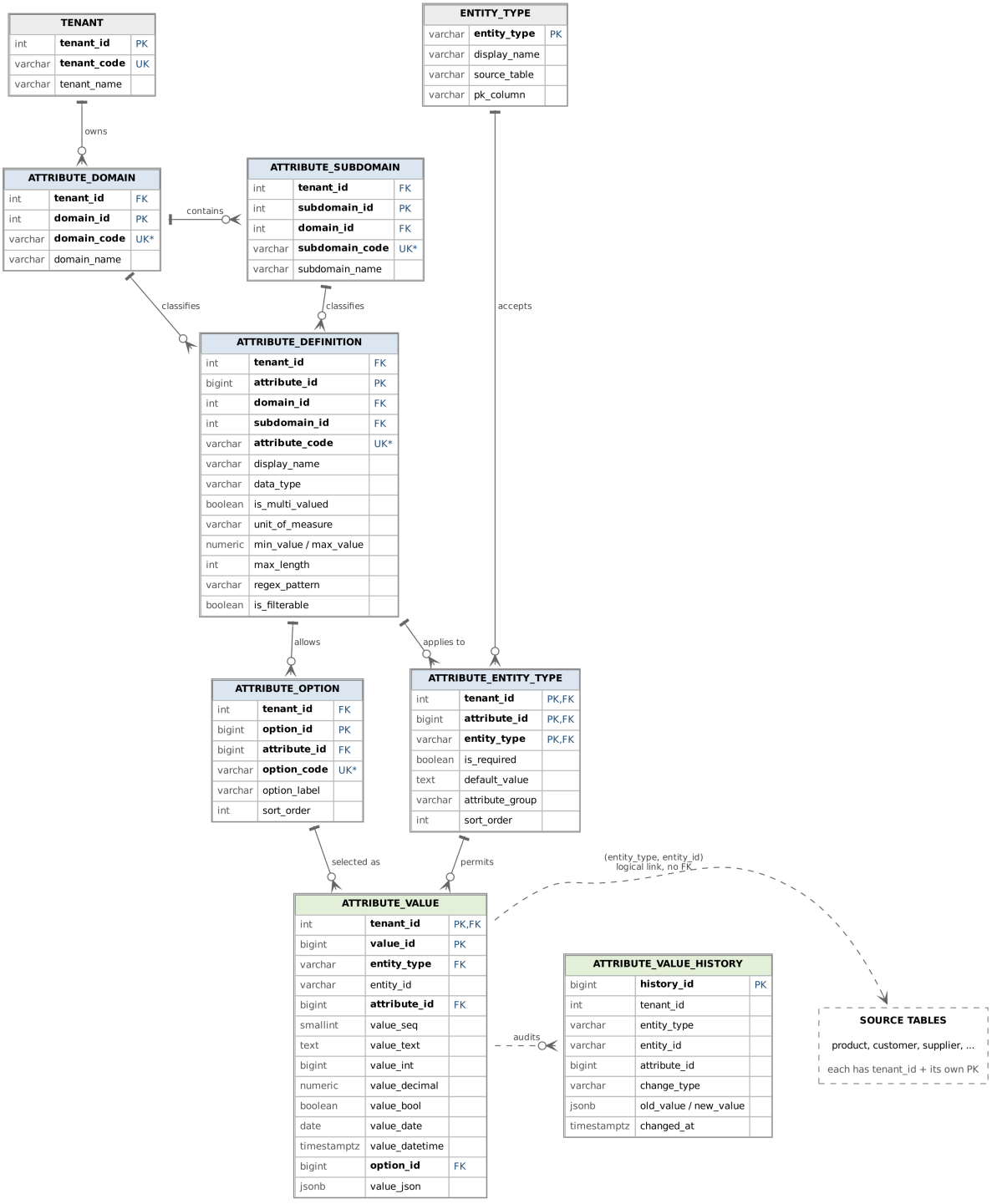


Figure 1. Tenant-aware custom attributes ERD. Not drawn: `tenant_id` on every tenant table references `tenant`; UK marks keys unique within the tenant.*

What changed from the original design

Original (product only)	New (tenant-aware, any entity)
`product_type`	`entity_type` (global list of tables) plus `attribute_domain` / `attribute_subdomain` (tenant-specific grouping)
`attribute_definition`, global	Same table plus `tenant_id`, `domain_id`, `subdomain_id`
`attribute_option`	Same table plus `tenant_id`
`product_type_attribute`	`attribute_entity_type`: which attributes each entity type may use, with required flag, default, UI group
`product_attribute_value` with `product_id`	`attribute_value` with `tenant_id`, `entity_type`, `entity_id`
JSONB cache column on `product`	`v_entity_attributes_json` view (works for every entity type)
Rules enforced by the application	Validation trigger, composite FKs and row-level security in the database

How the key pieces work

1. Domain and subdomain

A domain is a business area and a subdomain is a section inside it. They organize the catalog for admin screens, ownership and reporting, and they let the same attribute code exist in two areas: `region` in CRM and `region` in PURCHASING are separate attributes, because codes are unique per `(tenant_id, domain_id, attribute_code)`.

The subdomain is optional. When it is set, the composite foreign key `(tenant_id, domain_id, subdomain_id)` guarantees it belongs to the same tenant *and* the same domain as the attribute, so SALES > LOYALTY is rejected.

2. Entity type and entity ID

`entity_type` names the kind of record and `entity_id` holds its primary key as text, so it works for BIGINT keys (`product_id = 1` is stored as `'1'`), natural keys (`C-1001`) and UUIDs. If every source table uses a BIGINT key, change `entity_id` to BIGINT for smaller indexes and faster joins.

`attribute_entity_type` controls which attributes each entity type may use. `attribute_value` has a foreign key to it on `(tenant_id, attribute_id, entity_type)`, so writing a loyalty tier onto a product fails in the database, not just in the UI. One attribute can serve several entity types; in the sample, `region` applies to both customers and suppliers.

3. Tenant isolation, in three layers

- **Composite foreign keys.** Every reference includes `tenant_id`. A value can only point at its own tenant's attribute, and an option FK on `(tenant_id, attribute_id, option_id)` also proves the option belongs to *that* attribute, so a size cannot be stored as a color.
- **Tenant-leading keys.** `attribute_value` uses `PRIMARY KEY (tenant_id, value_id)`, and every index starts with `tenant_id`, so the table can be list- or hash-partitioned by tenant later without redesign.
- **Row-level security.** The application sets `app.tenant_id` per request, and a policy on every table filters on it. A forgotten `WHERE tenant_id` cannot leak data, and a session with no tenant set sees nothing.

4. Validation trigger

`trg_attribute_value_validate` closes the main weakness of EAV: the database checking values against their definition. On every insert and update it checks that:

- the filled column matches `data_type` (no text in an INTEGER attribute);
- single-valued attributes only use `value_seq = 1`;
- numbers respect `min_value` / `max_value`, and text respects `max_length` and `regex_pattern`;
- the attribute is active.

5. Write API, audit and orphan cleanup

- `set_attribute(tenant, entity_type, entity_id, domain, attribute, values[])` takes values as text, routes each into the right typed column (OPTION values by code), and upserts by sequence. Unchanged values are not rewritten; a real change is logged as one UPDATE.
- `trg_attribute_value_audit` writes old and new values to the history table.
- `fn_entity_attribute_cleanup` is a generic trigger attached to each source table. Deleting a product or customer deletes its attribute values, which replaces the `ON DELETE CASCADE` that a real foreign key would give.

Example data

Two tenants. ACME has SALES and CRM domains; BETA has its own SALES domain with its own *color* and options.

attribute_domain / attribute_subdomain

tenant	domain_code	subdomain_code
ACME	SALES	PRODUCT_SPECS, APPAREL
ACME	CRM	LOYALTY, CONTACT
BETA	SALES	APPAREL

attribute_definition (selected)

tenant	domain > subdomain	attribute_code	data_type	rules	entity types
ACME	SALES > PRODUCT_SPECS	ram_gb	INTEGER	GB, 1 to 1024	PRODUCT
ACME	SALES > PRODUCT_SPECS	screen_size	DECIMAL	inch, 5 to 40	PRODUCT
ACME	SALES	color	OPTION	multi-valued	PRODUCT (required)
ACME	SALES > APPAREL	size	OPTION		PRODUCT
ACME	CRM > LOYALTY	loyalty_tier	OPTION		CUSTOMER (required)
ACME	CRM > CONTACT	email	TEXT	regex, max 254	CUSTOMER (required)
ACME	CRM > CONTACT	region	TEXT	max 100	CUSTOMER, SUPPLIER
BETA	SALES > APPAREL	color	OPTION	multi-valued	PRODUCT (required)

attribute_value (selected, shown with option labels)

tenant	entity_type	entity_id	attribute_code	value_seq	typed column	value
ACME	PRODUCT	1	ram_gb	1	value_int	16
ACME	PRODUCT	1	color	1	option_id	Silver
ACME	PRODUCT	2	color	1	option_id	Black
ACME	PRODUCT	2	color	2	option_id	Silver
ACME	CUSTOMER	C-1001	loyalty_tier	1	option_id	Gold
ACME	CUSTOMER	C-1001	region	1	value_text	Florida
ACME	SUPPLIER	1	region	1	value_text	Florida
BETA	PRODUCT	4	color	1	option_id	Olive

Product 1 and supplier 1 share the entity ID *1* without conflict, because `entity_type` is part of the key.

Example queries

Write values through the API function

```
SELECT set_attribute(1, 'PRODUCT',  '1',      'SALES', 'ram_gb', ARRAY['16']);
SELECT set_attribute(1, 'PRODUCT',  '2',      'SALES', 'color',  ARRAY['BLK','SLV']);
SELECT set_attribute(1, 'CUSTOMER', 'C-1001', 'CRM',   'region', ARRAY['Florida']);
```

All attributes of one record

```
SELECT domain_code, subdomain_code, display_name, value_display, unit_of_measure
FROM   v_attribute_value
WHERE  tenant_id = 1 AND entity_type = 'PRODUCT' AND entity_id = '1';
```

Filter: products with RAM ≥ 16 GB and color Silver

```
SELECT p.product_id, p.sku
FROM   product p
WHERE  p.tenant_id = 1
  AND EXISTS (SELECT 1 FROM attribute_value v
              JOIN attribute_definition a ON a.attribute_id = v.attribute_id
              WHERE v.tenant_id = p.tenant_id AND v.entity_type = 'PRODUCT'
                AND v.entity_id = p.product_id::text
                AND a.attribute_code = 'ram_gb' AND v.value_int >= 16)
  AND EXISTS (SELECT 1 FROM attribute_value v
              JOIN attribute_definition a ON a.attribute_id = v.attribute_id
              JOIN attribute_option o     ON o.option_id = v.option_id
              WHERE v.tenant_id = p.tenant_id AND v.entity_type = 'PRODUCT'
                AND v.entity_id = p.product_id::text
                AND a.attribute_code = 'color' AND o.option_code = 'SLV');
```

Cross-entity search: everything in Florida

```
SELECT v.entity_type, v.entity_id
FROM   attribute_value v
JOIN   attribute_definition a ON a.attribute_id = v.attribute_id
WHERE  v.tenant_id = 1 AND a.attribute_code = 'region' AND v.value_text = 'Florida';
-- returns CUSTOMER C-1001 and SUPPLIER 1
```

Form metadata for a customer screen, grouped by domain and subdomain

```
SELECT dm.domain_code, sd.subdomain_code, aet.attribute_group,
       a.attribute_code, a.display_name, a.data_type, aet.is_required
FROM   attribute_entity_type aet
JOIN   attribute_definition a      ON a.attribute_id = aet.attribute_id
JOIN   attribute_domain dm         ON dm.domain_id   = a.domain_id
LEFT JOIN attribute_subdomain sd   ON sd.subdomain_id = a.subdomain_id
WHERE  aet.tenant_id = 1 AND aet.entity_type = 'CUSTOMER'
ORDER  BY aet.sort_order;
```

One JSON document per record

```
SELECT entity_type, entity_id, attributes
FROM   v_entity_attributes_json
WHERE  tenant_id = 1;
-- PRODUCT 2 -> {"color": ["BLK", "SLV"], "ram_gb": 12, "screen_size": 13.3}
```

The scripts also include a wide pivot, a missing-required-attributes check, an orphan check, a catalog inventory by domain, and an audit-trail query (`03_queries.sql`).

What the database enforces

Each rule below was tested; `04_validation_tests.sql` attempts every bad write and confirms it is rejected.

Bad write	Stopped by
Text such as "16GB" in an INTEGER attribute	Validation trigger (type check)
RAM of 4096 when the maximum is 1024	Validation trigger (min / max)
Invalid email format	Validation trigger (regex)
Two values for a single-valued attribute	Validation trigger (value_seq)

Bad write	Stopped by
Two typed columns filled in one row	CHECK `ck_av_one_value` and trigger
Option not in the list ("Slvr")	`set_attribute` lookup; FK for direct inserts
Another tenant's option, or another attribute's option	FK on `(tenant_id, attribute_id, option_id)`
Attribute not allowed for the entity type	FK to `attribute_entity_type`
Unknown entity type	FK to `attribute_entity_type` / `entity_type`
Subdomain from a different domain	FK on `(tenant_id, domain_id, subdomain_id)`
Reading or writing another tenant's rows	Row-level security policies
Values left behind when a record is deleted	Cleanup trigger on each source table
Required attribute missing	Not blocked on write; caught by the data-quality query

Pros

- **One attribute service for the whole application.** Products, customers, suppliers and any future table get custom attributes by adding one row to `entity_type`, with no new tables.
- **True multi-tenancy.** Each tenant owns its catalog, codes and options; isolation is enforced by keys and row-level security, not only by application code.
- **Organized catalog.** Domain and subdomain give admin screens, ownership and reporting a natural structure, and allow the same code in different business areas.
- **Stronger data quality than the original design.** Type, range, regex, option ownership and entity-type applicability are all checked inside the database.
- **Cross-entity queries.** One query can search every record type, for example everything in a region.
- **Clean audit trail.** Changes are recorded per attribute, and unchanged rewrites create no noise.
- **Partition-ready.** Tenant-leading keys allow partitioning by tenant when volume grows.

Cons

- **No real foreign key to the owning record.** `entity_id` cannot reference many tables at once. Cleanup triggers must be attached to every source table, and bulk loads or TRUNCATE can bypass them, so the orphan check should run on a schedule.
- **Text entity IDs.** Joins need `product_id::text`, and VARCHAR keys make larger indexes than BIGINT. Choose BIGINT if all source keys are numeric.
- **More joins and more moving parts.** Nine tables, three triggers and a policy per table take longer to build, test and explain than the product-only design.
- **Trigger cost on writes.** Each value write runs a definition lookup and an audit insert. This matters for bulk loads; those can disable triggers and run the validation as set-based queries instead.
- **Same EAV read costs.** Multi-attribute filters still need one EXISTS per condition, and BI tools still need pivoted views or wide tables.

- **Required attributes are checked after the fact,** because a record and its attributes are written in separate statements.
- **Session discipline for RLS.** Every connection must set `app.tenant_id`; connection pools must reset it between requests.

Design decisions and alternatives

Decision	Chosen	Alternative and when to prefer it
Link to the owning record	Polymorphic `(entity_type, entity_id)`	A central entity registry (as in the multi-domain NVP design) gives a real FK and cascade, at the cost of inserting a registry row for every record. Prefer it when referential integrity outweighs the extra write.
`entity_id` **type**	VARCHAR(100)	BIGINT when every source table has a numeric key: smaller indexes, no casts.
Attribute code uniqueness	Per tenant + domain	Per tenant only, if codes must be globally unambiguous within a tenant (simpler queries, no domain needed to resolve a code).
`entity_type` **scope**	Global (system)	Add `tenant_id` if tenants can register their own entity types.
Product sub-types	Not modeled; subdomain groups attributes	Add an `entity_subtype` column to `attribute_entity_type` if requiredness must differ between, for example, laptops and T-shirts.
Read cache	View `v_entity_attributes_json`	Materialize it, or keep a JSONB column on hot tables, when read volume justifies the sync cost.
Analytics	Pivot downstream	In Snowflake or another warehouse, pivot into wide tables or a VARIANT column per entity type; analysts should not query EAV directly.

Example scripts

Written for PostgreSQL 14+ and tested end to end on PostgreSQL 16. Run them in order:

Script	Contents
`01_schema.sql`	Tables, constraints, indexes, validation / audit / cleanup triggers, `set_attribute` function, two views, SQL Server and Snowflake notes.
`02_sample_data.sql`	Example source tables (product, customer, supplier) with cleanup triggers, two tenants, domains, subdomains, attributes, options, applicability and values.
`03_queries.sql`	Ten read patterns: record detail, multi-filter, pivot, cross-entity search, form metadata, catalog inventory, missing required, orphans, JSON per record, audit trail.
`04_validation_tests.sql`	Twelve tests: eleven bad writes that must be rejected and one valid write with orphan cleanup.
`05_row_level_security.sql`	`app_user` role, tenant policies on every table, and a demonstration of isolation.